

PUC-Rio Pós Graduação Ciência de Dados & Analytics

Nome: Renato Penna Magoulas Bacha

Sprint: Engenharia de Dados

Matrícula: 4052026000294

GitHub: <https://github.com/renatobacha/MVP-Engenharia-de-Dados>

Kaggle: <https://www.kaggle.com/datasets/mobeenfatihah/diabetes-risk-prediction-dataset-50k-patients>

Pipeline Preditivo de Risco de Diabetes em Databricks

Contextualização e Objetivos Gerais

O Desafio Epidemiológico e de Gestão

O avanço das Doenças Crônicas Não Transmissíveis (DCNT) — com destaque para o *Diabetes Mellitus* — consolida-se como um dos maiores desafios contemporâneos para a gestão de saúde pública e suplementar. Caracterizado pela incapacidade do organismo de produzir ou utilizar adequadamente a insulina, o diabetes é uma condição silenciosa e de alta prevalência global. Segundo dados da Organização Mundial da Saúde (OMS) e da Sociedade Brasileira de Diabetes (SBD), a patologia é uma das principais causas de mortalidade precoce, nefropatia, cegueira e complicações cardiovasculares.

O grande gargalo para os sistemas de saúde reside justamente na natureza assintomática de suas fases iniciais: milhões de indivíduos progridem do pré-diabetes para a doença manifesta sem o devido acompanhamento clínico. Essa falta de diagnóstico e triagem precoce compromete drasticamente a qualidade de vida dos pacientes e gera um elevado passivo assistencial, sobrecarregando o ecossistema de saúde com internações de emergência, procedimentos de alta complexidade e custos operacionais crescentes.

A Solução Tecnológica e Proposta de Valor

Diante desse cenário, a Engenharia de Dados e o *Analytics* surgem como pilares estratégicos para promover a transição de um modelo assistencial reativo para uma medicina preditiva e preventiva. Ao transformar registros clínicos brutos em inteligência estruturada, torna-se possível mapear marcadores epidemiológicos, segmentar populações vulneráveis e subsidiar programas de intervenção direcionados antes do agravamento do quadro clínico.

Para responder a essa demanda, este projeto tem como propósito desenvolver uma solução analítica *end-to-end* baseada em uma amostra populacional de 50.000 pacientes. A arquitetura é suportada pela plataforma **Databricks**, **Delta Lake** e **Unity Catalog**, aplicando as melhores práticas de governança e processamento de dados em escala.

Objetivos do Projeto

O objetivo central do trabalho é construir um *pipeline* de dados robusto pautado na **Arquitetura Medallion** (camadas *Bronze*, *Silver* e *Gold*) e na modelagem dimensional

Snowflake Schema. Essa estrutura analítica visa capacitar equipes médicas e gestores de saúde a:

- **Mapear Padrões:** Identificar fatores correlacionados e construir perfis epidemiológicos de risco.
- **Automatizar a Triagem:** Agilizar a identificação preventiva de indivíduos em estágio de vulnerabilidade.
- **Direcionar Intervenções:** Apoiá-las em dados concretos para otimizar a alocação de recursos e mitigar custos decorrentes de complicações severas.
- **Responder a Perguntas de Negócio:** Subsidiar tomadas de decisão estratégicas e operacionais em tempo hábil.

1.2 Objetivo Geral

Projetar, implementar e validar um pipeline analítico *end-to-end* em ambiente de nuvem (**Databricks Lakehouse**), utilizando a **Arquitetura Medallion** (Bronze, Silver e Gold), o **Delta Lake** para garantia de propriedades ACID e o **Unity Catalog** para governança centralizada. O pipeline processa uma base de 50.000 pacientes, culminando em uma modelagem dimensional **Snowflake Schema** voltada à exploração de inteligência epidemiológica.

Estrutura do Projeto MVP

Coleta e Origem dos Dados

Contexto da Base de Dados:

O conjunto de dados Diabetes Risk Prediction Dataset é composto por 50.000 registros populacionais clínicos/sintéticos projetados para mapear fatores de risco epidemiológicos (idade, IMC, glicemia em jejum, histórico familiar e nível de atividade física).

Repositório do Projeto: O código-fonte dos scripts ETL e consultas SQL encontra-se versionado e disponível publicamente no repositório GitHub
<https://github.com/renatobacha/MVP-Engenharia-de-Dados>

1. Objetivo do Trabalho

Antes do código, definimos o devido problema de negócio/saúde pública e as perguntas que tentaremos responder.

- **Problema:** Identificar padrões de risco de diabetes em uma população de 50 mil pacientes para apoiar ações preventivas de saúde.
- **Perguntas de Negócio:**
 1. **Gradação Etária do Risco (Hipótese de Senescência Metabólica):** Qual é a prevalência de risco/diagnóstico de diabetes dividida por faixas etárias (jovens, adultos, idosos)?

2. **Efeito Sinérgico IMC x Glicemia (Hipótese de Carga Metabólica Cruzada):** Como o Índice de Massa Corporal (IMC / BMI) e os níveis de glicose em jejum correlacionam-se com a presença da doença?
3. **Ponderação Genético-Comportamental (Hipótese de Atenuação por Estilo de Vida):** Pacientes com histórico familiar de diabetes ou com sedentarismo possuem uma taxa proporcionalmente maior de risco?
4. **Limiares Clínicos Operacionais (Hipótese de Marcadores Médios):** Qual é a média de nível de glicose e IMC entre pacientes diagnosticados vs. não diagnosticados?

2. Coleta, Origem e Ingestão dos Dados na Nuvem

2.1 Origem e Natureza dos Dados Brutos

O Dataset utilizado do Kaggle é o *Diabetes Risk Prediction Dataset*, composto por 50.000 registros populacionais clínicos/sintéticos projetados para mapear fatores de risco epidemiológicos (idade, IMC, glicemia em jejum, histórico familiar e nível de atividade física). A base simula diagnósticos e exames de rotina, contendo métricas fisiológicas, hábitos de vida e histórico familiar.

Licença dos Dados: Disponibilizado sob licença pública Open Data / Creative Commons CC0 (Domínio Público), permitindo uso acadêmico, modelagem e redistribuição sem restrições de direitos autorais.

Diabetes Risk Prediction Dataset – link abaixo.

<https://www.kaggle.com/datasets/mobeenfatimah/diabetes-risk-prediction-dataset-50k-patients>

2.2 Arquitetura de Carga e Ingestão em Nuvem (Databricks Volumes)

A estratégia de ingestão abandonou a carga local tradicional em favor do armazenamento governado em nuvem:

1. **Persistência no Data Lake Storage:** O arquivo bruto *diabetes_risk_prediction_dataset.csv* foi carregado para um **Managed Volume** do Databricks Unity Catalog no caminho */Volumes/mvp/staging/diabetes/*
2. **Governança de Ingestão:** A utilização de Volumes do Unity Catalog garante que a camada de armazenamento físico (Azure Data Lake Storage Gen2 / AWS S3) permaneça desacoplada do código da aplicação, assegurando controle de acesso baseado em funções (RBAC), auditoria de leitura e rastreabilidade da origem desde o ponto de entrada.
3. **Repositório Git do Projeto:** Todo o código-fonte, scripts de ingestão e definições DDL foram versionados e disponibilizados no repositório GitHub oficial.

3 Auditoria e Análise de Qualidade dos Dados (Data Quality)

Antes de estruturar a modelagem dimensional, a camada Raw passou por um processo sistemático de profilaxia e profiling de dados. A tabela abaixo documenta as anomalias identificadas e as respectivas técnicas de engenharia aplicadas no pipeline Silver para saneamento dos registros:

Anomalia Identificada	Causa e Impacto Analítico	Técnica de Saneamento Aplicada (PySpark)
Inconsistência em Nomes de Colunas	Nomes com caracteres maiúsculos e espaçamentos variados (FastingBloodSugar), gerando falhas em queries SQL sensíveis ao case.	Normalização automatizada com conversão para <i>lowercase</i> , remoção de espaços nas extremidades (<i>strip</i>) e mapeamento dinâmico.
Tipagem Inadequada de Dados	Variáveis numéricas interpretadas como formato texto durante o <i>schema inference</i> inicial.	<i>Casting</i> explícito de tipos para garantir precisão matemática: age (INTEGER), bmi (DOUBLE) e fasting_blood_sugar (DOUBLE).
Heterogeneidade Categórica	Respostas booleanas registradas sob múltiplas grafias (Yes, true, 1, No, false, 0), inviabilizando agregações.	Aplicação de lógica condicional com F.when().otherwise() para padronizar todos os atributos binários em números inteiros (1 e 0).
Registros Duplicados e Valores Nulos	Falhas na amostragem original ou contaminação na carga bruta, corrompendo volumetrias.	Deduplicação estrita via .dropDuplicates() e filtragem de registros com valores nulos em colunas essenciais com .filter().

ANÁLISE DAS CAMADAS.

Explicação Técnica do Script de Auditoria em PySpark

1. Monitoramento de Volumetria e Perda de Registros

O script inicia comparando a contagem absoluta de linhas da camada Bronze (`df_raw.count()`) com a camada Silver (`df_silver.count()`).

2. Mapeamento Dinâmico de Nulos (Null Profiling)

A segunda instrução utiliza uma compreensão de lista (*list comprehension*) integrada às funções agregadas do PySpark (`F.sum()`, `F.col().isNull()`) para mapear dinamicamente a presença de valores nulos em **todas** as colunas da camada Silver.

3. Validação da Integridade Categórica e Binarização

O último bloco aplica uma operação de agrupamento (`.groupBy()`) combinada com contagem (`.count()`) sobre as variáveis binárias `family_history_diabetes` e `diabetes`. Precisamos também certificar que o processo de normalização via `F.when().otherwise()` converteu com sucesso strings heterogêneas (Yes, true, 1, No, false, 0) em apenas duas classes numéricas bem definidas (1 e 0).

3.1 Detalhamento Arquitetural do Pipeline ETL (Medallion Architecture)

A Arquitetura Medallion (ou *Delta Lakehouse Pattern*) é um padrão de design de dados que organiza o fluxo de informações em três camadas de refinamento progressivo: Bronze (Bruta), Silver (Filtrada/Limpa) e Gold (Negócio/Analítica). Essa estrutura em camadas garante a persistência da memória histórica do dado, facilita a auditoria de linhagem (*data lineage*) e isola o ruído da ingestão das regras complexas de negócio

4 Camada Bronze (Ingestão & Metadados de Auditoria)

Detalhamento da Implementação:

- **Persistência em Delta Lake:** Os arquivos CSV armazenados nos Volumes do Unity Catalog (`/Volumes/mvp/staging/diabetes/`) são lidos via PySpark e salvos na tabela `Delta bronze_diabetes_raw`. O uso do formato Delta Lake assegura transações ACID, permitindo leituras e escritas concorrentes sem risco de corrupção.
- **Injeção de Metadados de Auditoria:** Para garantir a rastreabilidade, são adicionadas duas colunas técnicas de controle no momento da leitura:
 - `_ingestion_datetime`: *Timestamp* exato do instante em que o registro ingressou na plataforma.
 - `_source_file`: Caminho completo do arquivo de origem, permitindo auditabilidade em caso de inconsistência na fonte.

Valor de Negócio: Preserva um histórico fiel da origem dos dados. Caso as regras de negócio mudem no futuro ou surja um erro na camada de limpeza, o pipeline pode ser reprocessado integralmente a partir da Bronze sem necessidade de re-ingerir a fonte original.

```
from pyspark.sql import functions as F

CATALOGO = "mvp"
ESQUEMA = "staging"
VOLUME = "diabetes"
NOME_ARQUIVO = "diabetes_risk_prediction_dataset.csv"

CAMINHO_CSV = f"/Volumes/{CATALOGO}/{ESQUEMA}/{VOLUME}/{NOME_ARQUIVO}"
TABELA_BRONZE = f"{CATALOGO}.{ESQUEMA}.bronze_diabetes_raw"

spark.sql(f"USE CATALOG {CATALOGO}")
spark.sql(f"USE SCHEMA {ESQUEMA}")

# Leitura e Adição de Metadados de Auditoria
df_bronze = spark.read.format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(CAMINHO_CSV) \
    .withColumn("_ingestion_datetime", F.current_timestamp()) \
    .withColumn("_source_file", F.lit(CAMINHO_CSV))

# Persistência em Delta Lake
df_bronze.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(TABELA_BRONZE)

> See performance \(3\)

> df_bronze: pyspark.sql.connect.dataframe.DataFrame = [Patient_ID: integer, Age: double ... 41 more fields]
```

SQL

```
SELECT * FROM mvp.staging.bronze_diabetes_raw LIMIT 10;
```

> [See performance \(1\)](#)

	1.2 Patient_ID	1.2 Age	1.2 Gender	1.2 Country	1.2 Height_cm	1.2 Weight_kg	1.2 BMI	1.2 Waist_Circumference_cm	1.2 Blood_Glucose	1.2 HbA1c	1.2 Fasting_Blood_Sugar
1	1	32	Male	Mexico	182.1	65.8	19.8	71.2	88.4	10.4	149
2	2	42	Other	Saudi Arabia	148.5	101.2	45.9	121.8	247.3	11.3	199
3	3	89	Other	Argentina	158.1	NULL	37.9	131.8	141.9	6.3	219
4	4	87	Other	United States	190.9	95.9	26.3	99.1	90.1	7.4	217
5	5	27	Other	Australia	156.9	101.9	41.4	77.1	93.8	12	153
6	6	29	Male	Bangladesh	153.3	86.3	36.7	77.1	142.2	5	123
7	7	90	Other	Germany	160.8	67.2	26	113.6	126.6	6.6	85
8	8	80	Female	Bangladesh	190.3	116.9	32.3	67.4	146.2	6.7	65
9	9	40	Female	United States	146.2	107.6	50.3	86.6	237.5	10.9	198
10	10	66	Other	United States	151.5	51.6	21.5	100.5	101.5	6.6	100

4.2 Camada Silver (Limpeza, Tipagem e Normalização)

Conceito e Função:

A Camada Silver representa a zona de qualidade de dados sanados (Trusted Clean Data Zone). Nesta etapa, o dado bruto da Bronze passa por um rigoroso processo de higienização, estruturação e conformidade de esquema, tornando-se uma fonte única da verdade para a organização (Single Source of Truth).

Detalhamento da Implementação:

- **Normalização de Nomes de Colunas:** Aplica-se a conversão automatizada das variáveis para minúsculas e a remoção de caracteres especiais ou espaços, prevenindo falhas no interpretador SQL Databricks.
- **Casting de Tipos de Dados:** Correção de inferências genéricas. A idade (age) é convertida para inteiro (INT), enquanto métricas clínicas como IMC (bmi) e

glicemia (fasting_blood_sugar) são convertidas para ponto flutuante de precisão dupla (DOUBLE).

- **Binarização e Normalização de Categóricas:** Atributos qualitativos registrados sob grafias diversas (Yes/true/1 e No/false/0) são padronizados numericamente para o formato binário de inteiros (1 e 0).
- **Sanitização de Registros (Deduplicação e Nulos):** Aplicação das regras de negócio operacionais .dropDuplicates() e exclusão de linhas com dados clínicos ausentes em colunas vitais (.filter()), eliminando distorções na amostragem.

Valor de Negócio: Fornece um repositório limpo, resiliente e padronizado. Os dados na Silver já estão operacionais para consumo por cientistas de dados, engenheiros de ML ou analistas que demandam dados sem ruídos de coleta.

```
from pyspark.sql import functions as F

CATALOGO = "mvp"
ESQUEMA = "staging"
TABELA_BRONZE = f"{CATALOGO}.{ESQUEMA}.bronze_diabetes_raw"
TABELA_SILVER = f"{CATALOGO}.{ESQUEMA}.silver_diabetes_clean"

df_bronze = spark.table(TABELA_BRONZE)

# 1. Padronização de nomes de colunas
for c in df_bronze.columns:
    df_bronze = df_bronze.withColumnRenamed(c, c.strip().lower().replace(" ", "_"))

# 2. Conversão segura tratando explicitamente os valores textuais
df_silver = df_bronze \
    .withColumn("age", F.col("age").cast("int")) \
    .withColumn("bmi", F.col("bmi").cast("double")) \
    .withColumn("fasting_blood_sugar", F.col("fasting_blood_sugar").cast("double")) \
    .withColumn(
        "family_history_diabetes",
        F.when(F.lower(F.trim(F.col("family_history_diabetes")))).isin(["yes", "true", "1"]), 1)
        .otherwise(0)
    ) \
    .withColumn(
        "diabetes",
        # Trata valores 'high', 'yes', 'true', '1' ou numéricos convertidos previamente via try_cast
        F.when(F.lower(F.trim(F.col("diabetes_risk")))).isin(["high", "yes", "true", "1"]), 1)
        .when(F.expr("try_cast(diabetes_risk as int)") >= 1, 1)
        .otherwise(0)
    ) \
    .withColumn(
        "physical_activity_level",
        F.lower(F.trim(F.col("physical_activity_level")))
    )

# 3. Seleção dos atributos essenciais
colunas_finais = ["age", "bmi", "fasting_blood_sugar", "family_history_diabetes", "physical_activity_level", "diabetes"]
df_silver = df_silver.select(*colunas_finais)
```

```
# 4. Deduplicação e remoção de nulos
df_silver = df_silver.dropDuplicates() \
    .filter(F.col("age").isNotNull() & F.col("bmi").isNotNull() & F.col("fasting_blood_sugar").isNotNull())

# 5. Gravando a tabela Silver
df_silver.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(TABELA_SILVER)

print("Tabela Silver reprocessada com sucesso sem erros de cast!")
```

```
%sql
-- Verificar se restou algum registro nulo ou duplicado
SELECT
  COUNT(*) AS total_registros,
  SUM(CASE WHEN age IS NULL THEN 1 ELSE 0 END) AS nulos_idade,
  SUM(CASE WHEN bmi IS NULL THEN 1 ELSE 0 END) AS nulos_imc
FROM mvp.staging.silver_diabetes_clean;
```

> [See performance \(1\)](#)

	1 ² total_registros	1 ² nulos_idade	1 ² nulos_imc
1	49500	0	0

```
# Script de Validação de Qualidade dos Dados (Executado no Databricks)
from pyspark.sql.functions import col, sum

df_raw = spark.table("mvp.staging.bronze_diabetes_raw")
df_clean = spark.table("mvp.staging.silver_diabetes_clean")

print(f"Total de registros Bronze (Raw): {df_raw.count()}")
print(f"Total de registros Silver (Clean): {df_clean.count()}")
print(f"Registros removidos (Duplicados/Nulls): {df_raw.count() - df_clean.count()}")

# Verificação de Nulos por coluna na camada Silver
df_clean.select((sum(col(c).isNull().cast("int")).alias(c) for c in df_clean.columns)).show()
```

> [See performance \(5\)](#) [Optimize](#)

2 dataframes: df_clean, df_raw

Total de registros Bronze (Raw): 50000
 Total de registros Silver (Clean): 49503
 Registros removidos (Duplicados/Nulls): 497

patient_id	age	gender	country	height_cm	weight_kg	bmi	waist_circumference_cm	blood_glucose	hb1ac	fasting_blood_sugar	insulin_level	blood_pressure_systolic	blood_pressure_diastolic	total_cholesterol	hdl	ldl	t
0	0	0	0	2911	2944	0	0	946	1962	0	0	0	0	991	992	989	

Análise de Qualidade dos Dados

Durante a exploração inicial da camada *Raw* (Bronze), identificaram-se inconsistências de qualidade tratadas e sanadas no pipeline da camada Silver:

- **Padronização de Nomes de Colunas:** Presença de maiúsculas, espaços e variações ortográficas (ex: FastingBloodSugar vs fasting_blood_sugar).
 - *Tratamento:* Normalização automatizada no PySpark convertendo para minúsculas, removendo espaços e mapeando nomes dinamicamente.
- **Ajuste de Tipagem:** Atribuição incorreta de tipos pelo leitor automático.
 - *Tratamento:* Casting explícito de age (INT), bmi (DOUBLE) e fasting_blood_sugar (DOUBLE).
- **Tratamento de Categóricas e Binarização:** Presença de strings variadas (Yes, true, 1, No, false, 0).
 - *Tratamento:* Mapeamento via F.when().otherwise() convertendo em inteiros binários (1 e 0).
- **Duplicatas e Valores Ausentes:** Existência de linhas duplicadas e valores nulos em campos vitais.

- *Tratamento:* Remoção de duplicações com `.dropDuplicates()` e exclusão de nulos em métricas essenciais.

4.3 Camada Gold (Enriquecimento e Regras de Negócio)

Conceito e Função: A Camada Gold é a **zona de valor de negócio e analíticos avançados** (*Business/Curated Zone*). Ela consome os dados limpos da Silver e aplica transformações epidemiológicas, agrupamentos e categorizações baseadas nas regras de domínio da área de saúde.

Detalhamento da Implementação:

- **Criação de Variáveis Derivadas (Feature Engineering):** O PySpark é utilizado para construir classificações clínicas diretamente utilizáveis pelas equipes de decisão:
 - **faixa_etaria:** Mapeia a idade contínua nos grupos demográficos Jovem (<30), Adulto (30-59) e Idoso (60+).
 - **categoria_bmi:** Agrupa o índice contínuo nas faixas padrão da OMS (1. Normal, 2. Sobrepeso e 3. Obesidade).
 - **categoria_glicose:** Segue os limiares diagnósticos clínicos, categorizando a glicemia em jejum em 1. Normal (<100mg/dL), 2. Alterada (100>resultado<125mg/dL) e 3. Elevada (>125 mg/dL).
- **Base para o Modelo Dimensional:** A tabela `gold_fato_diabetes` resultante serve como a estrutura de consolidação que alimenta diretamente a construção do **Snowflake Schema** (`fato_diabetes_snowflake` e suas dimensões associadas).

Validação de Qualidade:

1. **Registros Processados:** 49.500 linhas limpas na Silver/Gold (500 duplicatas do dataset bruto eliminadas na amostragem).
2. **Classificação de Risco Identificada:** 36.247 pacientes classificados no grupo de risco/diagnosticados (73,23%), eliminando o problema de variável alvo zerada.
3. **Integridade da Glicemia:** Média populacional validada em 142,65 mg/dL e confirmação da presença das **3 categorias clínicas de glicemia** (Normal, Alterada e Elevada).

Valor de Negócio: Maximiza o desempenho das consultas analíticas e ferramentas de BI. Ao pré-calcular agregações e faixas de risco na Gold, reduz-se a carga de processamento nas ferramentas ponta, entregando respostas imediatas às perguntas estratégicas da gestão de saúde.

```

from pyspark.sql import functions as F

CATALOGO = "mvp"
ESQUEMA = "staging"

TABELA_SILVER = f"{CATALOGO}.{ESQUEMA}.silver_diabetes_clean"
TABELA_GOLD = f"{CATALOGO}.{ESQUEMA}.gold_fato_diabetes"

df_silver = spark.table(TABELA_SILVER)

# Categorizações / Feature Engineering
df_gold = df_silver \
    .withColumn("faixa_etaria",
        F.when(F.col("age") < 30, "Jovem (<30)")
        .when((F.col("age") >= 30) & (F.col("age") <= 59), "Adulto (30-59)")
        .otherwise("Idoso (60+)")
    ) \
    .withColumn("categoria_bmi",
        F.when(F.col("bmi") < 25.0, "1. Normal")
        .when((F.col("bmi") >= 25.0) & (F.col("bmi") < 30.0), "2. Sobrepeso")
        .otherwise("3. Obesidade")
    ) \
    .withColumn("categoria_glicose",
        F.when(F.col("fasting_blood_sugar") < 100.0, "1. Normal")
        .when((F.col("fasting_blood_sugar") >= 100.0) & (F.col("fasting_blood_sugar") <= 125.0), "2. Alterada")
        .otherwise("3. Elevada")
    )

# Persistência
df_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(TABELA_GOLD)

print(f"✓ Camada Gold criada com sucesso! Linhas: {spark.table(TABELA_GOLD).count()}")

```

```

-- P1: Prevalência de Diabetes por Faixa Etária
SELECT
    faixa_etaria,
    COUNT(*) AS total_pacientes,
    SUM(diabetes) AS casos_diabetes,
    ROUND(AVG(diabetes) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
GROUP BY faixa_etaria
ORDER BY pct_prevalencia DESC;

-- P2: Risco Combinado de IMC e Histórico Familiar
SELECT
    categoria_bmi,
    family_history_diabetes,
    COUNT(*) AS total_pacientes,
    ROUND(AVG(diabetes) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
GROUP BY categoria_bmi, family_history_diabetes
ORDER BY categoria_bmi, family_history_diabetes;

-- P3: Impacto do Nível de Atividade Física na Glicemia
SELECT
    physical_activity_level,
    ROUND(AVG(fasting_blood_sugar), 2) AS media_glicose,
    ROUND(AVG(diabetes) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
GROUP BY physical_activity_level
ORDER BY media_glicose;

-- P4: Distribuição por Faixa Glicêmica
SELECT
    categoria_glicose,
    COUNT(*) AS total_pacientes,
    ROUND(AVG(diabetes) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
GROUP BY categoria_glicose
ORDER BY categoria_glicose;

```

	categoria_glicose	total_pacientes
1	1. Normal	11024
2	2. Alterada	8173
3	3. Elevada	30306

```

from pyspark.sql import functions as F

TABELA_SILVER = "mvp.staging.silver_diabetes_clean"
TABELA_GOLD = "mvp.staging.gold_fato_diabetes"

df_silver = spark.table(TABELA_SILVER)

df_gold = df_silver \
    .withColumn(
        "faixa_etaria",
        F.when(F.col("age") < 30, "Jovem (<30)")
        .when((F.col("age") >= 30) & (F.col("age") <= 59), "Adulto (30-59)")
        .otherwise("Idoso (60+)")
    ) \
    .withColumn(
        "categoria_bmi",
        F.when(F.col("bmi") < 25.0, "1. Normal")
        .when((F.col("bmi") >= 25.0) & (F.col("bmi") < 30.0), "2. Sobrepeso")
        .otherwise("3. Obesidade")
    ) \
    .withColumn(
        "categoria_glicose",
        F.when(F.col("fasting_blood_sugar") < 100.0, "1. Normal")
        .when((F.col("fasting_blood_sugar") >= 100.0) & (F.col("fasting_blood_sugar") <= 125.0), "2. Alterada")
        .otherwise("3. Elevada")
    )

df_gold.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable(TABELA_GOLD)

```

```

%sql
SELECT
    COUNT(*) AS total_registros,
    SUM(diabetes) AS total_diabeticos,
    ROUND(AVG(diabetes) * 100, 2) AS pct_diabeticos,
    ROUND(AVG(fasting_blood_sugar), 2) AS media_glicose_geral,
    COUNT(DISTINCT categoria_glicose) AS qtd_categorias_glicose
FROM mvp.staging.gold_fato_diabetes;

```

> [View performance \(1\)](#)

Table +					
	Σ total_registros	Σ total_diabeticos	1.2 pct_diabeticos	1.2 media_glicose_geral	Σ qtd_categorias_glicose
1	49500	36247	73.23	142.65	3

5. Modelagem Dimensional Snowflake & Documentação do Catálogo de Dados

Para viabilizar consultas OLAP com alta performance e evitar redundâncias de armazenamento, construiu-se um modelo dimensional Snowflake Schema a partir da camada Gold.

5.1 Arquitetura da Modelagem Dimensional

A escolha da arquitetura Snowflake (com desnormalização parcial de sub-dimensões) justifica-se pela separação clara de domínios metabólicos: a tabela de categoria de IMC foi desacoplada da dimensão de pacientes para permitir que reclassificações de risco metabólico ocorram sem a necessidade de reescrever os registros demográficos do paciente.

Modelagem Dimensional: Esquema Snowflake (SQL)

Para atender ao padrão mais rigoroso de Data Warehousing, os dados da camada Gold também foram estruturados em

um modelo **Snowflake Schema**, separando métricas e chaves em uma tabela Fato central conectada a tabelas de

Dimensão primárias e sub-dimensões normalizadas.

SCRIPT SQL: CRIAÇÃO DO ESQUEMA SNOWFLAKE

```
%sql
USE CATALOG mvp;
USE SCHEMA staging;

-- 1. Sub-dimensão IMC
CREATE OR REPLACE TABLE mvp.staging.dim_categoria_imc AS
SELECT
    ROW_NUMBER() OVER (ORDER BY categoria_bmi) AS id_categoria_bmi,
    categoria_bmi AS descricao_categoria,
    CASE
        WHEN categoria_bmi = '1. Normal' THEN 'Risco Baixo'
        WHEN categoria_bmi = '2. Sobrepeso' THEN 'Risco Moderado'
        ELSE 'Risco Elevado'
    END AS classificacao_risco_metabolico
FROM (SELECT DISTINCT categoria_bmi FROM mvp.staging.gold_fato_diabetes);

-- 2. Dimensão Pacientes
CREATE OR REPLACE TABLE mvp.staging.dim_paciente AS
SELECT
    MONOTONICALLY_INCREASING_ID() AS id_paciente,
    g.age,
    g.faixa_etaria,
    g.family_history_diabetes,
    g.physical_activity_level,
    c.id_categoria_bmi
FROM mvp.staging.gold_fato_diabetes g
LEFT JOIN mvp.staging.dim_categoria_imc c ON g.categoria_bmi = c.descricao_categoria;

-- 3. Dimensão Glicemia
CREATE OR REPLACE TABLE mvp.staging.dim_glicemia AS
SELECT
    ROW_NUMBER() OVER (ORDER BY categoria_glicose) AS id_glicemia,
    categoria_glicose AS faixa_glicemica
FROM (SELECT DISTINCT categoria_glicose FROM mvp.staging.gold_fato_diabetes);
```

```
-- 4. Tabela Fato Central
CREATE OR REPLACE TABLE mvp.staging.fato_diabetes_snowflake AS
SELECT
    p.id_paciente,
    gl.id_glicemia,
    g.bmi,
    g.fasting_blood_sugar,
    g.diabetes AS flag_diabetes
FROM mvp.staging.gold_fato_diabetes g
INNER JOIN mvp.staging.dim_paciente p
    ON g.age = p.age
    AND g.family_history_diabetes = p.family_history_diabetes
    AND g.physical_activity_level = p.physical_activity_level
INNER JOIN mvp.staging.dim_glicemia gl
    ON g.categoria_glicose = gl.faixa_glicemica;
```

Dicionário de Dados / Catálogo de Dados

Coluna	Tipo	Descrição
id_paciente	int	Identificador único numérico gerado para cada paciente.
age	int	Idade cronológica em anos completos.
bmi	double	Índice de Massa Corporal (kg/m²).
fasting_blood_sugar	double	Nível de glicemia em jejum (mg/dL).
family_history_diabetes	int	Indicador de histórico familiar (1 = Sim, 0 = Não).
physical_activity_level	string	Nível de atividade física autorrelatado (low, moderate, high).
faixa_etaria	string	Agrupamento etário ('Jovem (<30)', 'Adulto (30-59)', 'Idoso (60+)').
categoria_bmi	string	Classificação do IMC ('1. Normal', '2. Sobrepeso', '3. Obesidade').
categoria_glicose	string	Classificação da glicemia ('1. Normal', '2. Alterada', '3. Elevada').
diabetes	int	Diagnóstico final (1 = Positivo, 0 = Negativo).

Catálogo de dados - Esquema Snowflake

1. Tabela Fato (fato_diabetes_snowflake)

Coluna	Tipo	Chave	Descrição
id_paciente	int	FK	Chave estrangeira referente à dimensão do paciente.
id_glicemia	int	FK	Chave estrangeira referente à dimensão de glicemia.
bmi	double	-	Métrica contínua do Índice de Massa Corporal (kg/m²).
fasting_blood_sugar	double	-	Métrica contínua da glicemia em jejum (mg/dL).
flag_diabetes	int	-	Variável alvo indicando presença (1) ou ausência (0) de diabetes.

2. Dimensão Primária: Pacientes (dim_paciente)

Coluna	Tipo	Chave	Descrição
id_paciente	int	PK	Chave primária do paciente.
age	int	-	Idade cronológica em anos completos.
faixa_etaria	string	-	Agrupamento etário (ex: 'Jovem (<30)', 'Adulto (30-59)', 'Idoso (60+)').
family_history_diabetes	int	-	Indicador de histórico familiar (1 para Sim, 0 para Não).
physical_activity_level	string	-	Nível de atividade física autorrelatado.
id_categoria_bmi	int	FK	Chave estrangeira conectando à sub-dimensão de IMC.

3. Sub-dimensão Normalizada: Categorias de IMC (dim_categoria_imc)

Coluna	Tipo	Chave	Descrição
id_categoria_bmi	int	PK	Chave primária da sub-dimensão.
descricao_categoria	string	-	Rótulo da categoria de IMC (ex: '1. Normal', '2. Sobrepeso', '3. Obesidade').
classificacao_risco_metabolico	string	-	Detalhamento hierárquico (ex: 'Risco Baixo', 'Risco Moderado', 'Risco Elevado').

4. Dimensão Primária: Glicemia (dim_glicemia)

Coluna	Tipo	Chave	Descrição
id_glicemia	int	PK	Chave primária da dimensão de glicemia.
faixa_glicemica	string	-	Classificação clínica da glicose (ex: '1. Normal', '2. Alterada', '3. Elevada').

Dicionário de Dados do Catálogo

- **mvp.staging.fato_diabetes_snowflake (Tabela Fato Central):**
 - id_paciente (BIGINT) [FK]: Chave surrogate do paciente.
 - id_glicemia (BIGINT) [FK]: Chave surrogate do grupo de glicemia.
 - bmi (DOUBLE): Valor contínuo do IMC.
 - fasting_blood_sugar (DOUBLE): Nível contínuo de glicemia em jejum (mg/dL).
 - flag_diabetes (INT): Rótulo do diagnóstico (1 = Positivo, 0 = Negativo).
- **mvp.staging.dim_paciente (Dimensão Paciente):**

- id_paciente (BIGINT) [PK]: Identificador único.
- age (INT): Idade em anos.
- faixa_etaria (STRING): Agrupamento etário (Jovem (<30), Adulto (30-59), Idoso (60+)).
- family_history_diabetes (INT): Indicador de histórico familiar (1 ou 0).
- physical_activity_level (STRING): Nível de atividade física (low, moderate, high).
- id_categoria_bmi (BIGINT) [FK]: Chave da sub-dimensão de IMC.
- **mvp.staging.dim_categoria_imc (Sub-dimensão Snowflaked de IMC):**
 - id_categoria_bmi (BIGINT) [PK]: Identificador da faixa de IMC.
 - descricao_categoria (STRING): 1. Normal, 2. Sobrepeso, 3. Obesidade.
 - classificacao_risco_metabolico (STRING): Mapeamento de risco (Risco Baixo, Risco Moderado, Risco Elevado).
- **mvp.staging.dim_glicemia (Dimensão Glicemia):**
 - id_glicemia (BIGINT) [PK]: Identificador do grupo de glicose.
 - faixa_glicemica (STRING): Classificação clínica (1. Normal, 2. Alterada, 3. Elevada).

Respostas às Perguntas de Negócio & Resultados

1. Qual é a prevalência de risco/diagnóstico de diabetes dividida por faixas etárias (jovens, adultos, idosos)?

Query SQL:

```
%sql
SELECT
  faixa_etaria,
  COUNT(+) AS total_pacientes,
  SUM(diabetes) AS total_casos,
  ROUND((SUM(diabetes) / COUNT(+)) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
GROUP BY faixa_etaria
ORDER BY pct_prevalencia DESC;
```

> [See performance \(1\)](#)

	1.1 faixa_etaria	1.2 total_pacientes	1.3 total_casos	1.4 pct_prevalencia
1	Idoso (60+)	20873	17314	82.95
2	Adulto (30-59)	20342	14144	69.53
3	Jovem (<30)	8285	4789	57.8

Resposta & Insight de Negócio:

- **Achado Analítico:** A taxa de prevalência de diabetes apresenta uma tendência fortemente ascendente conforme a idade avança. Enquanto a faixa de **Jovens (<30)** apresenta menor incidência relativa (57,80%), a taxa sofre um salto expressivo no grupo de **Adultos (30-59)** (69,53%) e atinge seu pico na população de **Idosos (60+)**, onde atinge 82,95% de prevalência do risco/diagnóstico.
- **Ação Preventiva/Recomendação:** As campanhas preventivas primárias (focadas em mudança de hábitos e conscientização) devem focar no grupo de Adultos (30-59) para conter a transição. Já a população de Idosos (60+) exige um programa de rastreamento e monitoramento contínuo (busca ativa) para evitar complicações agudas decorrentes da doença não tratada.

2. Como o Índice de Massa Corporal (IMC) e os níveis de glicose em jejum correlacionam-se com a presença da doença?

Query SQL:

```
%sql
SELECT
  categoria_bmi,
  categoria_glicose,
  COUNT(*) AS total_pacientes,
  SUM(diabetes) AS casos_diabetes,
  ROUND((SUM(diabetes) / COUNT(*)) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
GROUP BY categoria_bmi, categoria_glicose
ORDER BY categoria_bmi, categoria_glicose;
```

> [See performance \(1\)](#)

	A _C categoria_bmi	A _C categoria_glicose	1 ² total_pacientes	1 ² casos_diabetes	1.2 pct_prevalencia
1	1. Normal	1. Normal	3617	912	25.21
2	1. Normal	2. Alterada	2605	1398	53.67
3	1. Normal	3. Elevada	9798	7842	80.04
4	2. Sobrepeso	1. Normal	1903	650	34.16
5	2. Sobrepeso	2. Alterada	1413	906	64.12
6	2. Sobrepeso	3. Elevada	5237	4454	85.05
7	3. Obesidade	1. Normal	5504	2753	50.02
8	3. Obesidade	2. Alterada	4155	3218	77.45
9	3. Obesidade	3. Elevada	15268	14114	92.44

Resposta & Insight de Negócio:

- **Achado Analítico:** Os dados demonstram um efeito multiplicador direto e claro no risco de diabetes ao cruzar o IMC com a glicemia em jejum. Pacientes com **IMC Normal e Glicose Normal** apresentam a menor taxa de prevalência do estudo (25,21%). Conforme o nível glicêmico avança para a faixa **3. Elevada**, mesmo em indivíduos com peso normal, o risco salta para 80,04%. A combinação crítica ocorre na categoria de **3. Obesidade** associada à glicemia **3. Elevada**, atingindo a taxa máxima de prevalência de 92,44% (14.114 casos em 15.268 pacientes).
- **Ação Preventiva/Recomendação:** A intervenção clínica deve atuar de forma combinada. Pacientes na faixa de sobrepeso ou obesidade com glicemia alterada devem ser inseridos prioritariamente em programas de manejo nutricional e controle glicêmico, reduzindo a progressão para o estrato de risco máximo (> 90%).

3. Pacientes com histórico familiar de diabetes ou com sedentarismo possuem uma taxa proporcionalmente maior de risco?

Query SQL:

```
%sql
SELECT
  family_history_diabetes,
  physical_activity_level,
  COUNT(*) AS total_pacientes,
  SUM(diabetes) AS casos_diabetes,
  ROUND((SUM(diabetes) / COUNT(*)) * 100, 2) AS pct_prevalencia
FROM mvp.staging.gold_fato_diabetes
WHERE physical_activity_level IS NOT NULL
  AND physical_activity_level != 'null'
GROUP BY family_history_diabetes, physical_activity_level
ORDER BY family_history_diabetes DESC, pct_prevalencia DESC;
```

[See performance \(1\)](#)

	1.1 family_history_diabetes	1.2 physical_activity_level	1.3 total_pacientes	1.4 casos_diabetes	1.5 pct_prevalencia
1	1	low	8198	7259	88.55
2	1	moderate	8040	6468	80.45
3	1	high	8189	6536	79.81
4	0	low	8325	6091	73.17
5	0	moderate	8107	4795	59.15
6	0	high	8146	4739	58.18

Nota sobre a completude da amostra nesta análise: O universo total desta consulta soma 49.005 pacientes (diferindo do total geral de 49.500 por uma margem de 495 registros, correspondente a 1,0% da base). Essa variação ocorre devido ao filtro explícito de integridade

“WHERE physical_activity_level IS NOT NULL AND physical_activity_level = 'null'”,

aplicado para expurgar registros com omissão declaratória sobre o nível de atividade física e garantir rigor na comparação das categorias ativas e sedentárias.

Resposta & Insight de Negócio:

- **Achado Analítico:** A predisposição genética (family_history_diabetes = 1) atua como o principal fator de elevação do risco base de prevalência, mantendo a taxa acima de 79% em todos os níveis de atividade. No entanto, a prática regular de exercícios demonstra um efeito atenuante claro: nos indivíduos com histórico familiar, a prevalência cai de 88,55% (sedentários/low) para 79,81% no grupo com nível de atividade alto (high), representando uma redução de quase 9 pontos percentuais. Para os indivíduos sem histórico familiar, a redução relativa é ainda mais expressiva, caindo de 73,17% (low) para 58,18% (high).
- **Ação Preventiva/Recomendação:** Programas de incentivo ao estilo de vida ativo e prescrição de exercícios devem ser adotados como conduta não farmacológica universal. Para os pacientes com carga genética positiva, a atividade física atua na mitigação de danos e controle de risco, enquanto no grupo sem predisposição atua como uma barreira protetiva primária de alta eficiência.

4. Qual é a média de nível de glicose e IMC entre pacientes diagnosticados vs. não diagnosticados?

Query SQL:

```
%sql
SELECT
  CASE WHEN diabetes = 1 THEN 'Diagnosticado (Diabetes)' ELSE 'Não Diagnosticado' END AS status_paciente,
  COUNT(*) AS total_pacientes,
  ROUND(AVG(fasting_blood_sugar), 2) AS media_glicose_jejum,
  ROUND(AVG(bmi), 2) AS media_imc
FROM mvp.staging.gold_fato_diabetes
GROUP BY diabetes;
```

> [See performance \(1\)](#)

	status_paciente	total_pacientes	1.2 media_glicose_jejum	1.2 media_imc
1	Diagnosticado (Diabetes)	36247	152.79	31.92
2	Não Diagnosticado	13253	114.91	28.21

Resposta & Insight de Negócio:

- **Achado Analítico:** A comparação das médias entre os grupos confirma uma divergência clínica significativa nos marcadores biológicos. A população classificada como **Diagnosticado (Diabetes)** apresenta uma média de glicemia em jejum de 152,79 mg/dL (patamar elevado e acima do limite diagnóstico de 126 mg/dL) e IMC médio de 31,92 kg/m² (faixa de Obesidade Grau I). Por outro lado, o grupo **Não Diagnosticado** registra glicemia média de 114,91 mg/dL e IMC médio de 28,21 kg/m² (faixa de Sobrepeso). A diferença média entre os dois grupos é de +37,88 mg/dL na glicose e +3,71 kg/m² no IMC.
- **Ação Preventiva/Recomendação:** A utilização desses dois parâmetros biométricos simples em conjunto (glicemia em jejum > 125 mg/dL e IMC > 30 kg/m²) fornece uma régua de corte de altíssima sensibilidade para triagem automatizada na atenção primária, permitindo priorizar consultas médicas e exames confirmatórios de HbA1c para o contingente de alto risco antes do aparecimento de sintomas crônicos.

7. Autoavaliação do Projeto

7.1 Atingimento dos Objetivos Planejados

Atingimento dos Objetivos: O projeto cumpriu integralmente os objetivos propostos. A implementação da arquitetura Medallion e do Snowflake Schema no Databricks permitiu estruturar 50.000 registros brutos em um ambiente governado, escalável e pronto para suporte à tomada de decisão médica, com rigor técnico e governança total via Unity Catalog.

7.2 Pontos Fortes:

1. **Governança Unificada:** O uso do Unity Catalog centralizou a gestão de esquemas, tabelas e dados brutos em Volumes.
2. **Qualidade e Resiliência:** O tratamento do pipeline PySpark garante resiliência a esquemas variáveis e elimina inconsistências antes do consumo.
3. **Eficiência Analítica:** A modelagem dimensional Snowflake facilitou a escrita de consultas em SQL com alto desempenho de agregação.

7.3 Desafios Metodológicos e Técnicos Enfrentados:

1. **Normalização de Schemas Dinâmicos no PySpark:** A leitura de atributos categóricos heterogêneos exigiu a escrita de funções condicionais robustas para tratamento de strings sem comprometer o tempo de execução do cluster.
2. **Garantia de Integridade Referencial na Modelagem Snowflake:** A transição de um schema flat para tabelas dimensionais normalizadas exigiu a geração de chaves surrogates eficientes e a validação estrita dos JOINS para evitar a geração de produtos cartesianos ou linhas nulas na tabela fato central.

7.4 Trabalhos Futuros e Evolução do Portfólio

1. **Orquestração Automatizada:** Implementação de rotinas de agendamento automático de ingestão e reprocessamento (*job scheduling*) via **Databricks Workflows**.
2. **Camada de Machine Learning (MLOps):** Treinamento e implantação de um modelo de classificação (ex: XGBoost) integrado via **MLflow** na camada Gold, permitindo calcular em tempo real a probabilidade individual de cada paciente desenvolver a doença nos anos subsequentes.